EQUIPO 4	
García Cerda Randal Guillermo (2112493 ITS) Sarmiento Hernández Cesar Martín (2119427 ITS) Torres Pasillas Angela Susana (2101844 ITS).	
Nombre del curso: Componentes de Lógica de Negocios	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad: 3	**Actividad**: Diseño arquitectural del sistema
Fecha: 21/05/2026	
Bibliografia (Formato APA): Presentaciones de clase.	

Objetivo

En esta actividad convertirán el análisis de negocio elaborado en la Unidad 2 en una propuesta de arquitectura de software. Trabajarán con la misma empresa y el mismo caso. La inteligencia artificial podrá apoyar el diseño y la revisión, pero cada decisión deberá justificarse con evidencia del proyecto

Procedimiento

EVIDENCIA DE PRUEBAS AUTOMATIZADAS

Proyecto VetCare 24

Este documento presenta la evidencia de las pruebas automatizadas del proyecto VetCare 24. Las pruebas verifican los comportamientos programados en los escenarios que cubre la suite: reglas de negocio del modelo de dominio, coordinación de los casos de uso CU-01 a CU-05, límites transaccionales (Unit of Work), consultas del lado Query de CQRS, permisos por rol y coherencia entre el diseño (diagrama de clases y decisiones ADR-01 a ADR-05) y el código.

1. Entorno de ejecución

Elemento	Valor
Sistema operativo	Windows
Editor	Visual Studio Code (terminal integrada de PowerShell)
Lenguaje	Python 3.13.7
Marco de pruebas	unittest (biblioteca estándar de Python)

2. Procedimiento

Las pruebas se ejecutan en la terminal integrada de Visual Studio Code, desde la carpeta raíz del repositorio, que contiene las carpetas src/ y tests/. El comando ejecutado es:

```
py -m unittest discover -s tests -t . -v
```

La opción discover localiza los módulos test_*.py de la carpeta tests, la opción -t . fija la raíz del repositorio como directorio de nivel superior (necesario para que los módulos importen tests.entorno) y la opción -v muestra el nombre y el resultado de cada prueba. El archivo tests/__init__.py agrega la carpeta src/ a la ruta de búsqueda de Python, por lo que no se requiere instalar el paquete vetcare24.

3. Entorno de pruebas (tests/entorno.py)

Todas las suites usan la clase Entorno, que arma servicios reales con infraestructura en memoria (AlmacenEnMemoria) y espías que dejan una bitácora ordenada de las llamadas importantes. Esto permite verificar el orden de las operaciones sin depender de una base de datos ni de servicios externos.

Componente	Función en las pruebas
`UnitOfWorkEspia`	Registra uow.iniciar, uow.confirmar y uow.revertir en la bitácora y expone el estado de la transacción.
`ControlEspia`	Registra acceso.verificar para comprobar que la autorización es el primer paso de cada caso de uso.
`NotificadorEspia`	Registra notificador.notificar y permite simular que el servicio de notificación no está disponible.
`Entorno._sembrar()`	Siembra la mascota M-001 (Firulais), su expediente EXP-001 y los esquemas de vacunación Rabia (validado), Parvovirus (no validado) y Moquillo (sin esquema).
`Entorno.huella()`	Resume el estado persistido para comparar antes y después de una operación y demostrar que no hubo cambios.
Usuarios de prueba	Propietario, Recepción, Veterinario (VET-01) y Personal clínico.

4. Suites de pruebas

La carpeta tests del proyecto contiene seis módulos de prueba, además del módulo auxiliar entorno.py. La cantidad de pruebas de cada uno se obtuvo contando los métodos test_* definidos en cada archivo.

Archivo	Pruebas	Qué comprueba
`test_reglas_de_negocio.py`	16	Reglas protegidas por el dominio, sin aplicación ni infraestructura: periodo de cita coherente con la duración del servicio, rechazo de horarios traslapados, horarios contiguos y de otro día, evento CitaAgendada, notas clínicas con autor y sin operaciones de edición o borrado, seguimiento de vacunación con esquema validado, no validado o ausente, y emergencias sin cita que requieren motivo.

Archivo	Pruebas	Qué comprueba
`test_casos_de_uso.py`	16	Recorrido de los casos de uso CU-01 a CU-05 con flujos normales y de rechazo: solicitar cita, generar nota clínica, registrar vacunación (el esquema no validado no es error y deja seguimiento pendiente), registrar hospitalización con seguimiento continuo y atender emergencia sin cita previa.
`test_permisos_por_rol.py`	7	Matriz de permisos por rol (provisional, P7): Recepción administra citas, solo el Veterinario registra notas clínicas y vacunaciones, la hospitalización la inician Veterinario o Personal clínico, la verificación ocurre antes de abrir la transacción y toda operación tiene reglas de acceso definidas.
`test_transacciones_y_eventos.py`	8	Unit of Work (ADR-02): reversión ante fallas de persistencia, confirmación al éxito, autorización antes de la transacción, un agregado modificado por caso de uso; notificación al veterinario después de confirmar, notificación pendiente sin deshacer el registro y eventos devueltos solo si la operación se completa.
`test_consultas_cqrs.py`	6	Lado Query de CQRS (ADR-03): las consultas no modifican el estado, no abren transacción y devuelven View Models; la agenda se ordena por hora y une la mascota; el expediente solo lo consulta personal autorizado; el handler no depende de repositorios ni de Unit of Work.
`test_coherencia_diseno.py`	7	Coherencia entre el diseño y el código: regla de dependencias entre capas (ADR-01, ADR-02), dominio solo con biblioteca estándar, existencia de los componentes del diagrama de clases, un Repository con su implementación por interfaz, servicios que dependen de interfaces, un servicio por caso de uso e importación de todos los módulos.
Total	**60**	

5. Trazabilidad con el análisis de negocio

Las pruebas citan en su documentación las reglas, necesidades y decisiones que protegen, lo que permite justificar cada decisión de diseño con evidencia del proyecto.

Referencia	Prueba(s) que la cubren
R-01 a R-04, R-09, R-14	test_permisos_por_rol.py y test_consultas_cqrs.py (permisos por rol y acceso a información clínica).
R-05, NB-06, NB-07, Q3	test_reglas_de_negocio.py (periodo coherente y horarios traslapados).
R-02, R-03, NB-12	test_reglas_de_negocio.py (nota con autor y expediente sin edición ni borrado).
R-06, NB-19, P3	test_reglas_de_negocio.py y test_casos_de_uso.py (emergencia sin cita ni mascota registrada).
R-08, NB-18	test_casos_de_uso.py (seguimiento continuo desde el ingreso hospitalario).
R-10	test_casos_de_uso.py (no se cita a una mascota no registrada).
R-11, R-12, R-13, NB-14, NB-16	test_casos_de_uso.py y test_reglas_de_negocio.py (historial de vacunación y esquema validado).
CR-8	test_transacciones_y_eventos.py (notificación después de confirmar).
ADR-01 a ADR-05	test_coherencia_diseno.py, test_transacciones_y_eventos.py, test_consultas_cqrs.py, test_permisos_por_rol.py y test_casos_de_uso.py.
US07	test_consultas_cqrs.py (consulta de citas programadas sin modificar estado).
P6, P7	test_transacciones_y_eventos.py (eventos de dominio) y test_permisos_por_rol.py (matriz provisional de permisos).

6. Resultados

Indicador	Resultado
Pruebas definidas en la suite	60
Pruebas ejecutadas	60
Pruebas aprobadas	60
Pruebas fallidas	0

Indicador	Resultado
Pruebas con error	0
Pruebas omitidas	0
Tiempo registrado	0.079 s
Resultado final	OK

En unittest, la línea final «OK» sin anotaciones adicionales indica que no hubo fallas, errores ni pruebas omitidas.

7. Evidencia visual

La figura 1 corresponde a la captura de Visual Studio Code con el comando ejecutado, la parte final de la salida y el resumen.

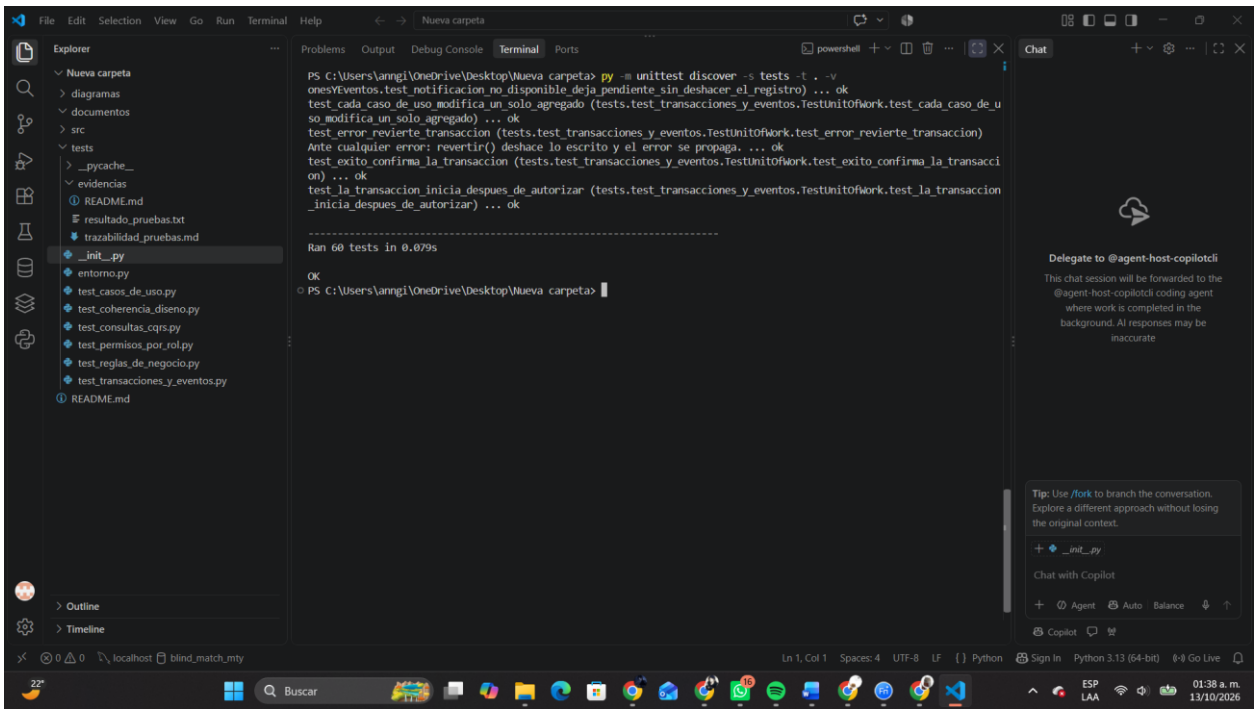


Figura 1. Ejecución de las pruebas automatizadas del proyecto VetCare 24

8. Interpretación y limitaciones

El resultado acredita que los escenarios programados en la suite se comportan como esperan sus aserciones en la versión ejecutada. No demuestra por sí solo que exista una aplicación gráfica

completa, que funcionen integraciones externas reales ni que estén cubiertos todos los casos posibles.

Las pruebas usan infraestructura en memoria y espías, no una base de datos ni un servicio de notificación reales. Además, algunos supuestos están marcados como provisionales en las propias pruebas: la matriz de permisos por rol (P7), la fórmula de fechas del refuerzo de vacunación (pendiente de la Unidad 2) y la emergencia sin mascota registrada (P3). Los resultados no sustituyen la revisión de las demás etapas de la actividad ni implican que todas estén aprobadas.

9. Conclusión

La ejecución de py -m unittest discover -s tests -t . -v sobre las seis suites de VetCare 24 terminó con 60 pruebas ejecutadas, 60 aprobadas y resultado OK en 0.079 segundos. Esta evidencia respalda la validación del código para los escenarios cubiertos por las suites y sirve como